



Không chỉ là "Có mạng không?"

Network Thực Chiến · Series: Debug Mạng từ A–Z · Tập 01

Nội dung

1. **ICMP là gì?** — Giao thức nền tảng của ping
2. **Cấu trúc gói tin ICMP** — Đọc được từng byte
3. **ping hoạt động thế nào** — Bên dưới lệnh đơn giản
4. **TTL Fingerprinting** — Đoán OS từ TTL
5. **MTU Discovery** — Debug VPN/Tunnel bí ẩn
6. **Cheatsheet** — Các cờ thực chiến
7. **ICMP Tunneling** — Khi ping trở thành kênh bí mật
8. **Phát hiện & Phòng thủ**

Phần 1

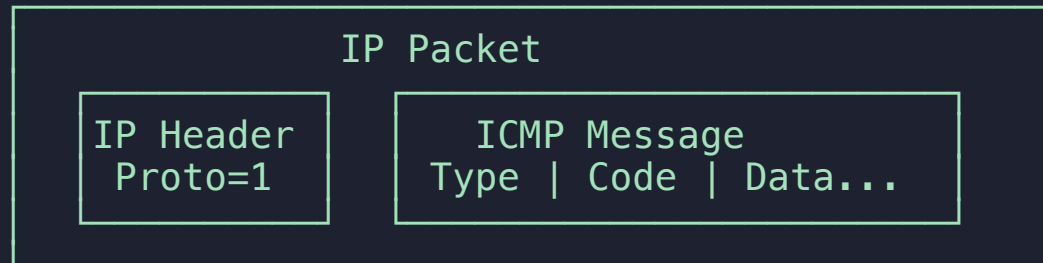
ICMP Protocol

ICMP là gì?

ICMP = Internet Control Message Protocol (RFC 792, 1981)

*Giao thức **điều khiển và báo lỗi** của tầng Network (L3).*

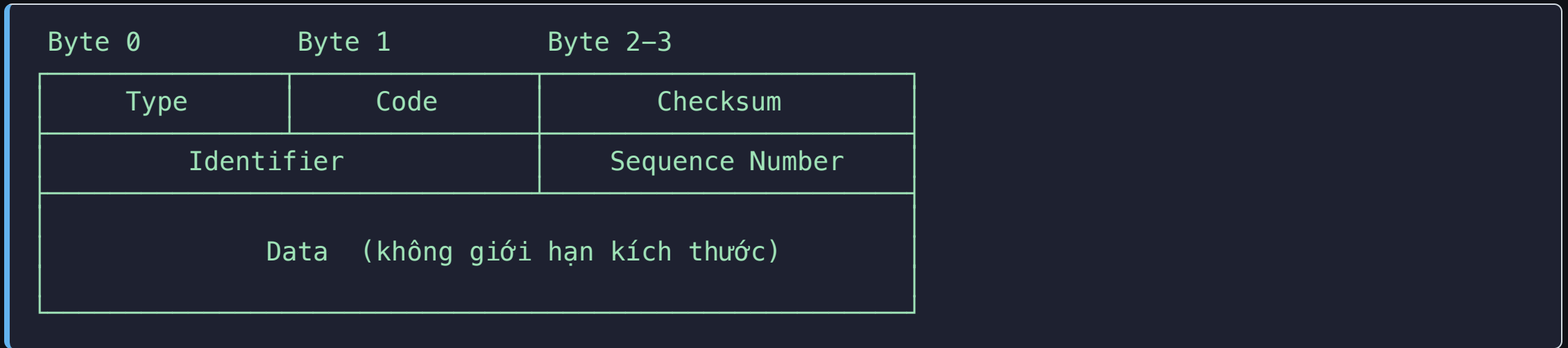
*`ping` chỉ dùng **2 trong số 40+ loại message** của ICMP.*



ICMP **không** là TCP/UDP — không có port, không có connection state.

Đây là lý do firewall thường xử lý ICMP riêng biệt.

Cấu trúc Header ICMP



Trường	Size	Ý nghĩa
Type	1 byte	Loại ICMP message
Code	1 byte	Chi tiết trong từng Type
Checksum	2 bytes	Kiểm tra toàn vẹn
Identifier	2 bytes	Khớp Request ↔ Reply
Sequence	2 bytes	<code>icmp_seq</code> trong output ping
Data	Tùy ý	Payload — chứa được bất cứ thứ gì

Các ICMP Type quan trọng

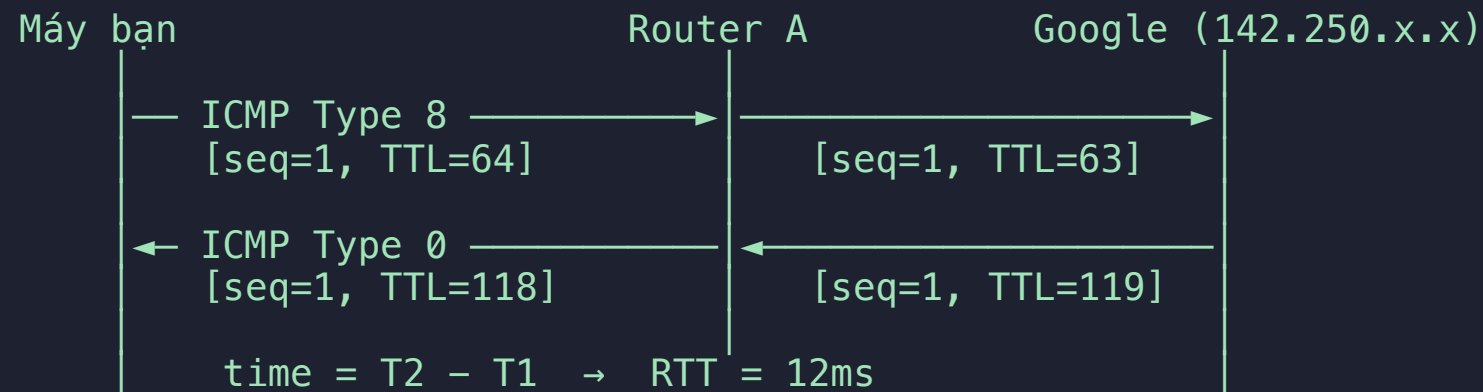
Type	Code	Tên	Gặp khi nào
0	0	Echo Reply	Phản hồi ping
3	0	Net Unreachable	Route không tồn tại
3	1	Host Unreachable	Host không reach được
3	3	Port Unreachable	UDP port đóng
3	4	Frag Needed (DF set)	MTU mismatch — rất quan trọng
8	0	Echo Request	Gói ping gửi đi
11	0	TTL Exceeded	traceroute/mtr dùng điều này
11	1	Fragment Timeout	Reassembly thất bại

💡 Khi ping báo "**Frag needed**" → đó là Type 3 Code 4.
Khi mtr thấy từng hop → đó là Type 11 Code 0 từ mỗi router.

Phần 2

ping hoạt động thể nào

Luồng gói tin của ping



TTL giảm 1 tại mỗi router → khi về đến máy bạn, TTL đã bị trừ đi số hop đã đi qua.

Đọc output ping

```
PING google.com (142.250.196.46): 56 data bytes
64 bytes from 142.250.196.46: icmp_seq=0 ttl=118 time=12.465 ms
64 bytes from 142.250.196.46: icmp_seq=1 ttl=118 time=11.832 ms
64 bytes from 142.250.196.46: icmp_seq=3 ttl=118 time=13.102 ms

--- google.com ping statistics ---
4 packets transmitted, 3 packets received, 25.0% packet loss
round-trip min/avg/max/stddev = 11.832/12.466/13.102/0.518 ms
```

Dấu hiệu	Ý nghĩa
icmp_seq nhảy (1 → 3)	Gói seq=2 bị mất
packet loss > 0%	Mạng có vấn đề — dùng <code>mtr</code> để tìm hop
time cao bất thường	Congestion hoặc routing vòng
Request timeout	ICMP bị block hoặc host down

Payload mặc định của ping

Wireshark / tcpdump hiển thị Data field của ICMP ping chuẩn:

```
0000  08 00 f3 2a  00 01 00 01  ← Type=8, Code=0, Checksum, ID, Seq
0008  10 11 12 13  14 15 16 17  ← Data: pattern lặp lại CÓ THỂ DỰ ĐOÁN
0010  18 19 1a 1b  1c 1d 1e 1f
0018  20 21 22 23  24 25 26 27
0020  28 29 2a 2b  2c 2d 2e 2f
0028  30 31 32 33  34 35 36 37
```

⚠ Khi payload **không** có pattern `10 11 12 13...` này
→ dấu hiệu đầu tiên của **ICMP Tunneling**

Phần 3

TTL Fingerprinting & MTU Discovery

TTL Fingerprinting

Mỗi OS có **TTL mặc định khác nhau** khi gửi gói tin:

OS / Thiết bị	TTL gửi đi (mặc định)
Linux / macOS	64
Windows	128
Cisco IOS / Network devices	255

Công thức: TTL gốc = TTL nhận về + số hop đã đi qua

```
ping google.com
# 64 bytes from 142.250.196.46: icmp_seq=0 ttl=118 time=12.4 ms
#                                     ^^^
# TTL=118 → 128 - 118 = 10 hop → Máy chủ chạy Windows
```

Ứng dụng: Biết OS của thiết bị trung gian giúp đặt đúng giả thuyết về cấu hình firewall, MTU mặc định, và hành vi TCP.

MTU Discovery — Debug VPN/Tunnel

MTU (Maximum Transmission Unit) = kích thước tối đa gói tin. Ethernet = **1500 bytes**.

Qua VPN/Tunnel, MTU giảm do overhead header:

```
Ethernet MTU: 1500B
└ WireGuard overhead: ~60B → MTU thực = 1440B
└ GRE overhead: ~24B → MTU thực = 1476B
└ VXLAN overhead: ~50B → MTU thực = 1450B
```

Cách tính payload test:

```
Payload = MTU - IP header (20B) - ICMP header (8B)
          = 1500 - 20 - 8 = 1472 bytes
```

Nếu MTU bị cắt mà không biết: TCP chậm bí ẩn, UDP bị drop ngẫu nhiên.

ICMP Type 3 Code 4 (**Frag Needed**) sẽ được gửi về — nhưng thường bị firewall chặn mất!

MTU Discovery — Thực hành

```
# Linux: -s = payload size, -M do = đặt DF bit (cầm phân mảnh)
```

```
ping -s 1472 -M do 192.168.1.1
```

```
# ✅ Thành công → MTU >= 1500
```

```
ping -s 1473 -M do 192.168.1.1
```

```
# ❌ "Frag needed and DF set" → MTU < 1500, cần giảm
```

```
# Binary search để tìm chính xác MTU:
```

```
ping -s 1412 -M do 10.0.0.1 # ✅
```

```
ping -s 1413 -M do 10.0.0.1 # ❌ → MTU thực = 1440 (1412+20+8)
```

```
# macOS: dùng -D thay vì -M do
```

```
ping -s 1472 -D 192.168.1.1
```

💡 Nếu tìm ra MTU = 1440 (VPN WireGuard), cần set **MSS Clamp = 1400** cho TCP để tránh TCP silent retransmit vô tận.

Phần 4

Cheatsheet Thực chiến

Các cờ quan trọng

```
# Giới hạn số gói (không ping vô tận)
ping -c 10 google.com

# Tăng tần suất – phát hiện packet loss nhanh hơn
sudo ping -i 0.1 google.com          # 10 gói/giây

# Flood ping – stress test (CHỈ TRONG LAB)
sudo ping -f -c 10000 192.168.1.1

# Gắn timestamp – vẽ biểu đồ latency spike
ping -D google.com
# [1714012345.123456] 64 bytes from ...: icmp_seq=1 time=12ms

# Chỉ định source interface – test policy routing
ping -I eth1 8.8.8.8                  # Linux
ping -I 192.168.2.1 8.8.8.8          # Linux (dùng IP)

# Quiet mode – chỉ lấy summary (dùng trong script)
ping -c 100 -q google.com
```


Phần 5

ICMP Tunneling

Khi ping trở thành kênh bí mật

Ý tưởng cốt lõi

Trường **Data** của ICMP không bị kiểm soát bởi giao thức.

Firewall thường cho ICMP đi qua **mà không inspect payload**.

Gói ICMP bình thường:

IP Hdr	ICMP Header	Data: 10 11 12 13 14 15 ...
--------	-------------	-----------------------------

▲ Pattern vô nghĩa

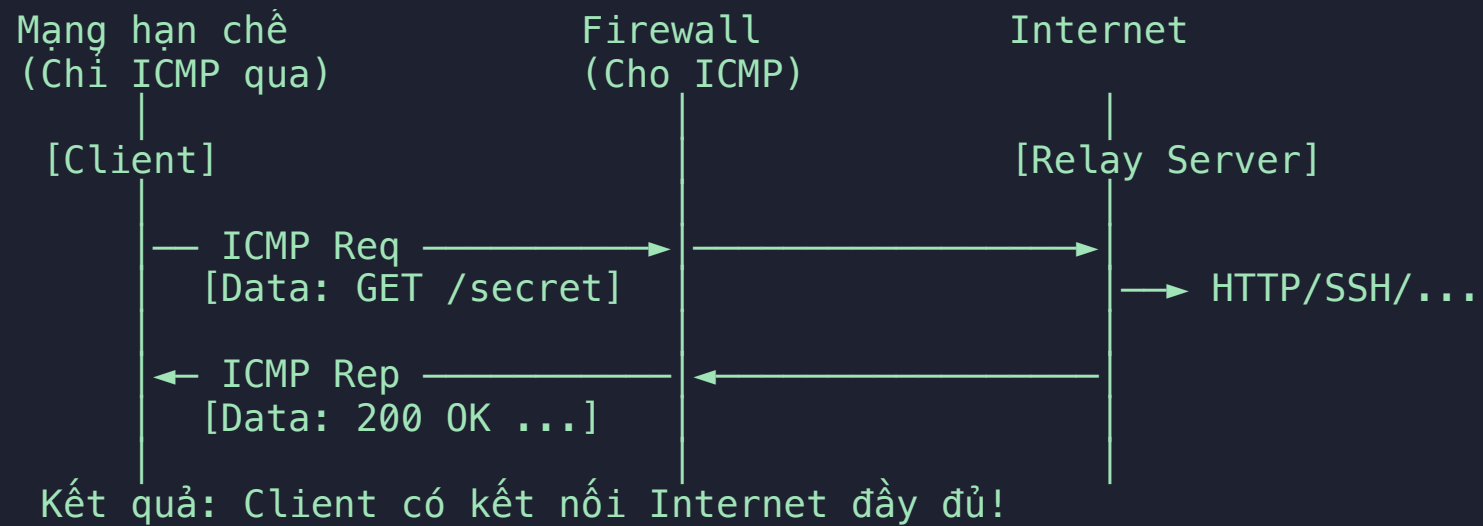
Gói ICMP Tunneling:

IP Hdr	ICMP Header	Data: [TCP/SSH packet nhúng]
--------	-------------	------------------------------

▲ Traffic thật bị giấu trong đây!

Kết quả: Bypass hoàn toàn firewall chỉ cho phép ICMP.

Cơ chế hoạt động



Công cụ ICMP Tunneling

Công cụ	Chức năng	Dấu hiệu
ptunnel-ng	Tunnel TCP qua ICMP, có auth	Payload lớn cố định (1024B+)
icmptunnel	Nhúng IP packet vào ICMP Data	IP header thứ 2 trong Data
hans	Full VPN qua ICMP Echo	Tần suất rất cao, payload random
nping	Custom ICMP packet tùy ý	Payload bất kỳ

Phát hiện ICMP Tunneling

Chỉ số	Ping bình thường	ICMP Tunneling
Payload size	56–64 bytes	Lớn bất thường (200B–64KB)
Tần suất	1 gói/giây	Hàng trăm gói/giây
Payload entropy	Thấp (pattern lặp)	Cao (~8.0, random/encrypted)
Identifier	Tăng dần theo PID	Cố định hoặc random
Request/Reply size	Bằng nhau	Bất đối xứng

Phát hiện — tcpdump & Wireshark

tcpdump:

```
# Xem ICMP payload dạng hex
sudo tcpdump -nn -i any icmp -X

# Lọc ICMP payload lớn bất thường
sudo tcpdump -nn "icmp and greater 150"

# Đếm tần suất ICMP theo src IP
sudo tcpdump -nn icmp | awk '{print $3}' \
  | cut -d. -f1-4 | sort | uniq -c | sort -rn
```

Wireshark filter:

```
icmp && frame.len > 150
icmp && !(icmp.type == 0 || icmp.type == 8)
```

💡 Payload ICMP tunneling thường có **entropy cao** (gần 8.0 bits/byte).

Wireshark: Statistics → Capture File Properties → Entropy

Phòng thủ

```
# Rate-limit ICMP Echo (iptables)
iptables -A INPUT -p icmp --icmp-type echo-request \
  -m limit --limit 10/s -j ACCEPT
iptables -A INPUT -p icmp --icmp-type echo-request -j DROP

# Giới hạn ICMP payload size (nftables)
nft add rule inet filter input \
  ip protocol icmp icmp type echo-request \
  ip length > 100 drop

# Block hoàn toàn nếu không cần ping từ ngoài
iptables -A INPUT -p icmp -j DROP
```

Chiến lược phòng thủ:

- Kiểm tra payload size — ICMP > 100 bytes là bất thường
- Monitor tần suất ICMP per-source
- Deep packet inspection: payload có IP header nhúng không?
- IDS/IPS rule: Snort `alert icmp any any -> any any (dsize:>200; msg:"Large ICMP payload");`

ICMP Tunneling trong thực tế

*ICMP Tunneling là kỹ thuật phổ biến trong **APT (Advanced Persistent Threat)** để duy trì kênh **C2 (Command & Control)** qua mạng kiểm soát nghiêm ngặt.*

Các tình huống thực tế:

- Bypass captive portal tại sân bay, khách sạn
- Exfiltrate data qua firewall chỉ cho ICMP outbound
- Duy trì persistence trong môi trường Zero Trust
- Red team operations: test detection capability của SOC

Nguyên tắc: Hiểu attack vector → Viết detection rule chính xác hơn.

Tổng kết

Key Takeaways

Kỹ năng	Lệnh cần nhớ
Connectivity cơ bản	<code>ping -c 10 host</code>
TTL Fingerprinting	Đọc TTL → tính OS nguồn
MTU Discovery	<code>ping -s 1472 -M do host</code>
Flooding / stress test	<code>sudo ping -f -c 10000 host</code>
Phát hiện ICMP tunnel	<code>tcpdump "icmp and greater 150"</code>

Khi nào dùng công cụ khác?

Nếu muốn...	Dùng
Biết lỗi ở hop nào	<code>mtr</code>
Test port cụ thể	<code>nc -zv host port</code>
Đo bandwidth	<code>iperf3</code>
Xem gói tin thực	<code>tcpdump</code>
Debug DNS	<code>dig</code>

Tài liệu thêm

- **RFC 792** — ICMP specification gốc
- **RFC 1122** — Requirements for Internet Hosts
- **ptunnel-ng** — ICMP tunnel tool (GitHub)
- **Series Debug Mạng từ A-Z** → `../debug-mang-az/README.md`

Cảm ơn đã theo dõi! 🙏

Network Thực Chiến

Tập tiếp theo: **mtr** — Chẩn đoán đường đi mạng

"Đừng đoán mò — hãy đo."